

THE REDEMPTION PLATFORM

Cloud Engineer Technical Assessment Submission

Author: Myint Hein Kyaw

Table of Contents

1. Executive Summary
2. Architecture Diagram
3. Solution Overview
4. AWS Well-Architected Mapping
5. RTO / RPO Matrix
6. Risk Assessment Matrix
7. Cost Optimization Strategy
8. Team Responsibility Matrix
9. Trade-off Analysis
10. Conclusion

1. Executive Summary

The Redemption Platform is a cloud-native hotel loyalty point redemption system built on AWS. The architecture uses Terraform, Amazon EKS, Aurora PostgreSQL, Redis, ArgoCD, Argo Rollouts, KEDA, Karpenter, Prometheus, Alertmanager, AWS Backup, and Velero to provide a scalable, secure, and highly available platform.

2. Architecture Diagram

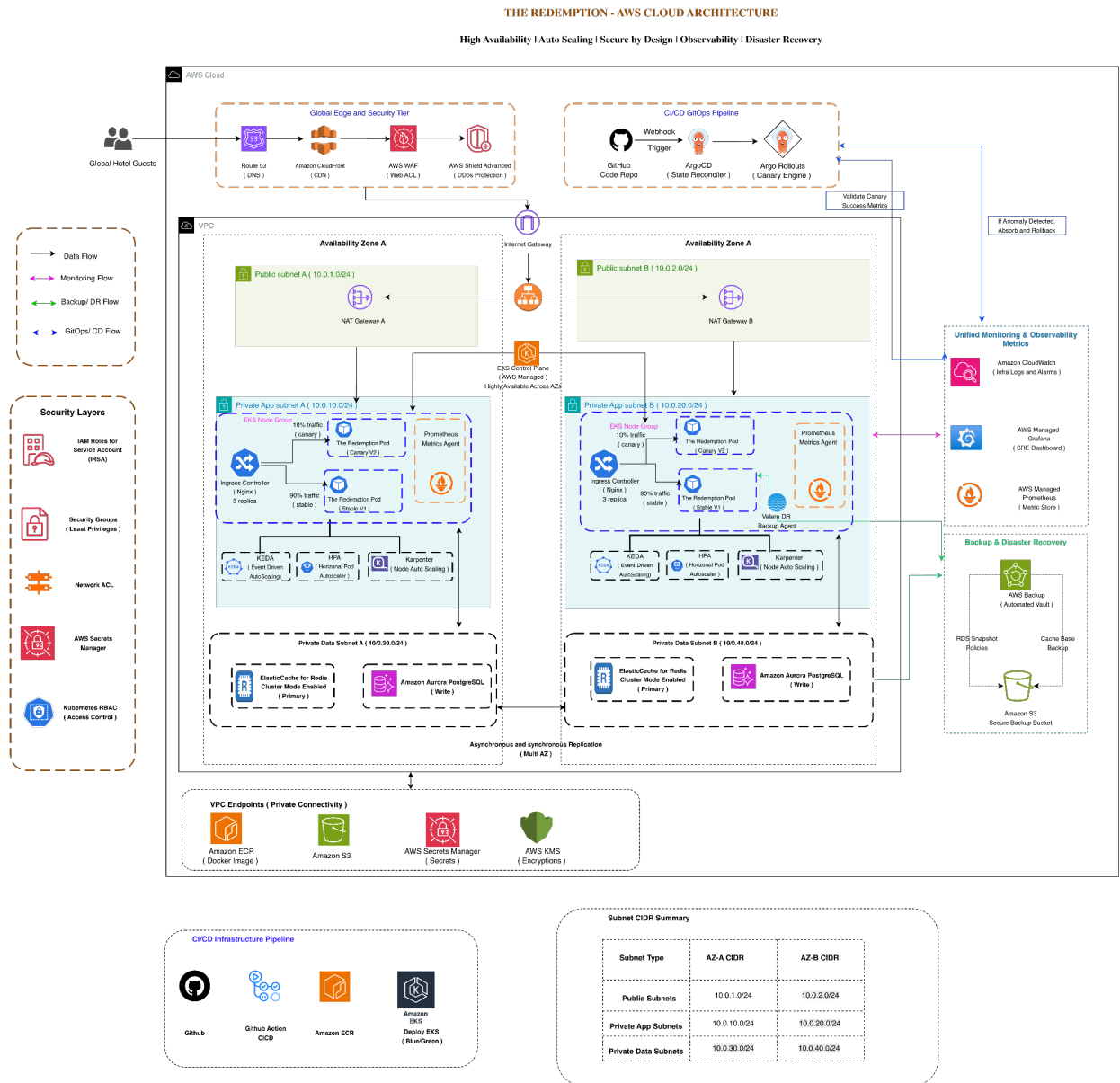


Figure 1: The Redemption Platform AWS Cloud Architecture

3. Solution Overview

User traffic enters through CloudFront and AWS WAF before reaching the Application Load Balancer. Applications run on Amazon EKS across multiple Availability Zones. Aurora PostgreSQL provides the transactional database layer and Redis provides caching. GitOps is implemented through ArgoCD and Argo Rollouts. Monitoring is delivered through Prometheus, Alertmanager, Amazon Managed Prometheus, and Grafana.

4. AWS Well-Architected Mapping

Pillar	Implementation
Operational Excellence	Terraform, GitOps, Automation
Security	WAF, Security Groups, Secrets, KMS
Reliability	Multi-AZ, Aurora, Autoscaling, Backups
Performance Efficiency	Redis, EKS, KEDA, Karpenter
Cost Optimization	Aurora Serverless v2, Spot Capacity, Consolidation

5. RTO / RPO Matrix

Component	RTO	RPO
Application	30 min	6 hrs
Database	1 hr	6 hrs
Kubernetes Resources	30 min	6 hrs

6. Risk Assessment Matrix

Risk	Impact	Likelihood	Mitigation
------	--------	------------	------------

AZ Failure	High	Low	Multi-AZ deployment
Traffic Spike	High	Medium	KEDA + Karpenter
Deployment Failure	Medium	Medium	Argo Rollouts
Data Loss	High	Low	AWS Backup + Velero

7. Cost Optimization Strategy

Aurora Serverless v2, Redis caching, CloudFront caching, autoscaling, Spot instances, and Karpenter node consolidation are used to optimize operational costs.

8. Team Responsibility Matrix

Role	Count	Responsibilities
Senior Cloud Engineer	1	Architecture, Reviews, Security
Junior Engineer	2	Operations, Monitoring, Deployments

9. Trade-off Analysis

Decision	Alternative	Benefit	Trade-off
Karpenter	Cluster Autoscaler	Fast scaling	Extra complexity
Argo Rollouts	Rolling Update	Safer releases	Additional controller
KEDA	HPA only	Event-driven scaling	More components

10. Conclusion

The architecture satisfies assessment requirements for scalability, reliability, security, observability, GitOps, and disaster recovery while following modern cloud-native engineering practices.